MAE 3780: Final Report

Ava Rosenow (ahr74), Bella Pensabene (irp28),
Maya Chakraborty (dac399)

1. **Robot Design and Strategy Overview**

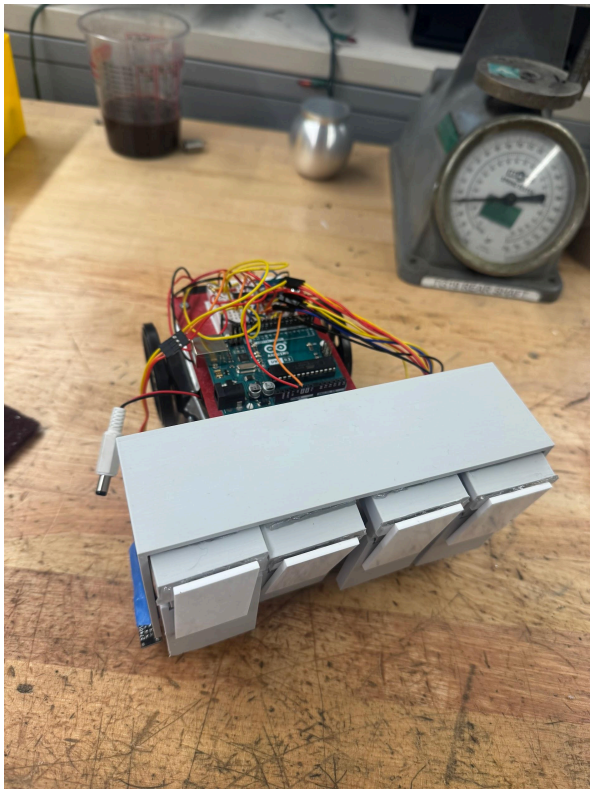
a) An overview of your robot's mechanical, electrical, and software design:

Our robot consisted of a collection box and _4_ one-way flaps that allowed cubes to enter but not exit. We used two QTI sensors on either side of our robot to detect if the robot's front edge had detected a border. The code would check the color reading of the QTIs and the color sensor, and if they detected a border, would rotate the robot in the appropriate direction and then have it keep driving forward, making an effective infinite loop sweep of the board.

b) How does the robot work and what does it (ideally) do during the competition?:

The robot gathers cubes by sweeping them and then trapping the cubes inside the collection box. It starts at an angle and then moves forward until it reaches a black border. Once it does, it uses the QTI sensors to detect which direction to turn and moves away from the border.

c) Include a single picture of your robot in this section.



2. Design Process Reflection

a) A description of your group's design/prototyping/testing process throughout the course of the project:

We started our design process with a rough idea of what we wanted our robot to have. We decided on one-way flaps but had different ideas of how to execute it. Some ideas included having small flaps that could fold into itself or using a flexible material. We prototyped these ideas with cardboard and decided to 3D print hinges. After assembling our robot, we went to lab sessions to test our robot on the mat and check that the robot could successfully pick up cubes. We adjusted our code to make sure that the robot would stay within the black borders.

b) How did your group progress through the design process and achieving the milestones?:

Our group definitely had different experiences completing the various milestones. Starting with the first two, we didn't encounter much difficulty. Although the code was a bit difficult to work out at first, getting them done along with our design presentation proved not to be a large challenge, and we stayed aligned throughout the process on what our vision for the robot was. We did, however, encounter a roadblock at milestone three, which we were unable to complete on time (though we did achieve it before milestone four). We had difficulty both with the code in general and with the fact that none of our schedules aligned during that period, with exams, assignments, and extracurricular activities combined, so we were each working individually, but couldn't find a time to work together. While that experience was definitely disappointing as a team, we used it as a learning experience, and not only completed milestone four on time, but also collaborated on the final robot design and code over the weekend to come up with a product we were all proud of.

c) Did you run into any significant roadblocks, need to change the design of your robot, etc.?

There were no significant roadblocks with our robot other than those detailed above. We originally planned on using ultrasonic sensors to detect the blocks and move towards them. However, while working on milestones 3 and 4, we realized that the ultrasonic sensors added unnecessary complexity to the robot. Starting the

robot at an angle and turning around at the borders would make the robot sweep the board itself.

3. Competition Analysis

Our robot achieved 1 win and 2 ties on competition day. It was successfully able to gather at least 1 cube at every match. The hinges were successful, and they were useful when the robot at the borders, since it would keep the blocks inside the robot even when the robot would reverse. However, the collection box was not big enough to gather more than 5 cubes. This would prevent the cubes from properly being sweeped inside and when the robot would reverse, it would leave the cubes outside the collection box. It would also often get stuck at the corners when it would sense the black at both QTI sensors, causing it to get stuck at the border.

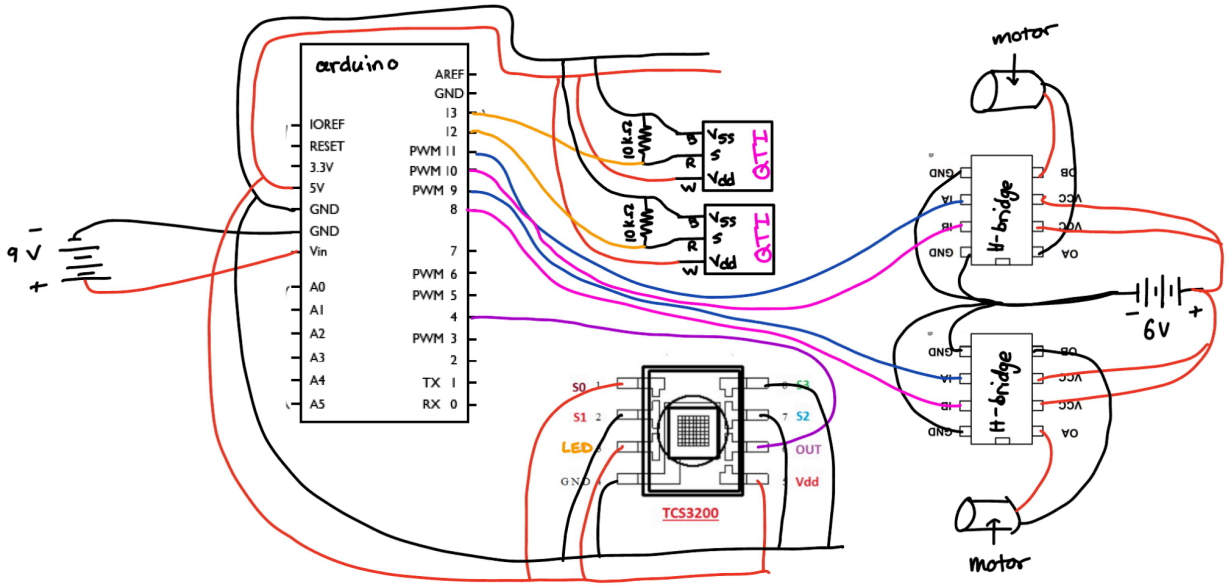
4. Conclusions

We believe that the robot was semi-successful. With minor changes to the design (making the collection box wider) and the code (accounting for sensing in both QTI sensors), the robot would be more successful at competition.
We would suggest to students completing the project next year to start all milestones early, keep a very open line of communication, and make sure you have more than one team member troubleshooting at all times. Additionally, be creative; the best part of the competition was seeing all of the different design ideas we never would have thought of, even if they weren't performing the best.

Appendix A

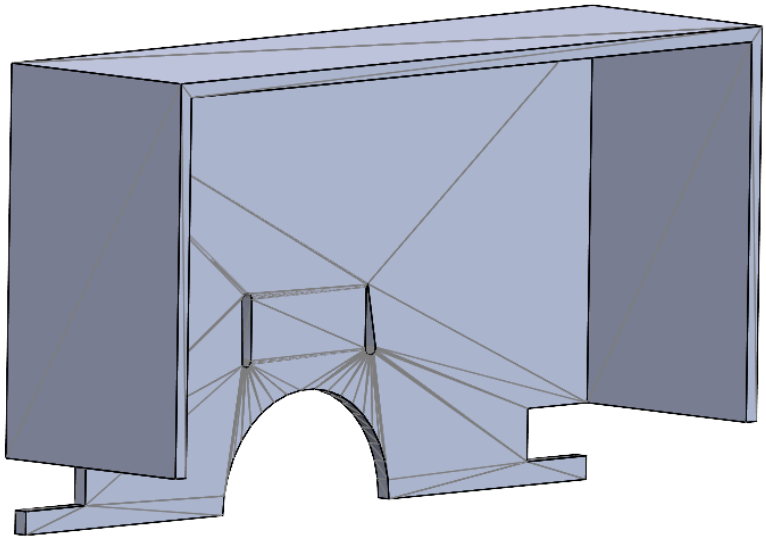
BOM		
Part	**Unit**	**Cost**
QTI Sensor	2	4.00
3D Print	N/A	30.30
Laser Cut	N/A	5.50
	Total:	**$39.80**

Appendix B

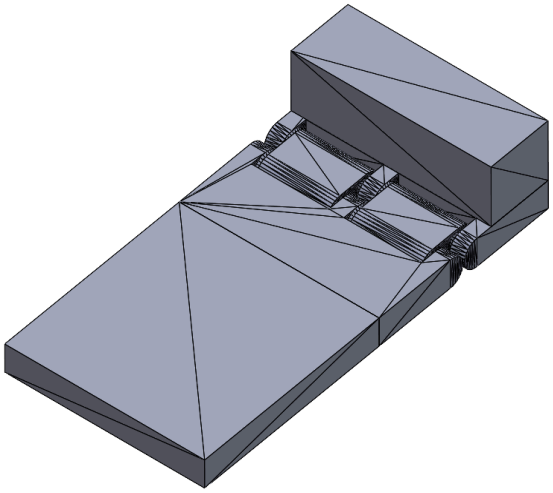

arduino
AREF
GND
13
IOREF
12
RESET
PWM 11
3.3V
PWM 10
5V
PWM 9
GND
8
GND
Vin
7
PWM 6
A0
PWM 5
A1
4
A2
PWM 3
A3
2
A4
TX 1
A5
RX 0
9 V
10kΩ
10kΩ
Vss
S
R
Vdd
OUT
S0
S1
LED
GND
S3
S2
OUT
Vdd
TCS3200
motor
H-bridge
H-bridge
6V
motor

Appendix C

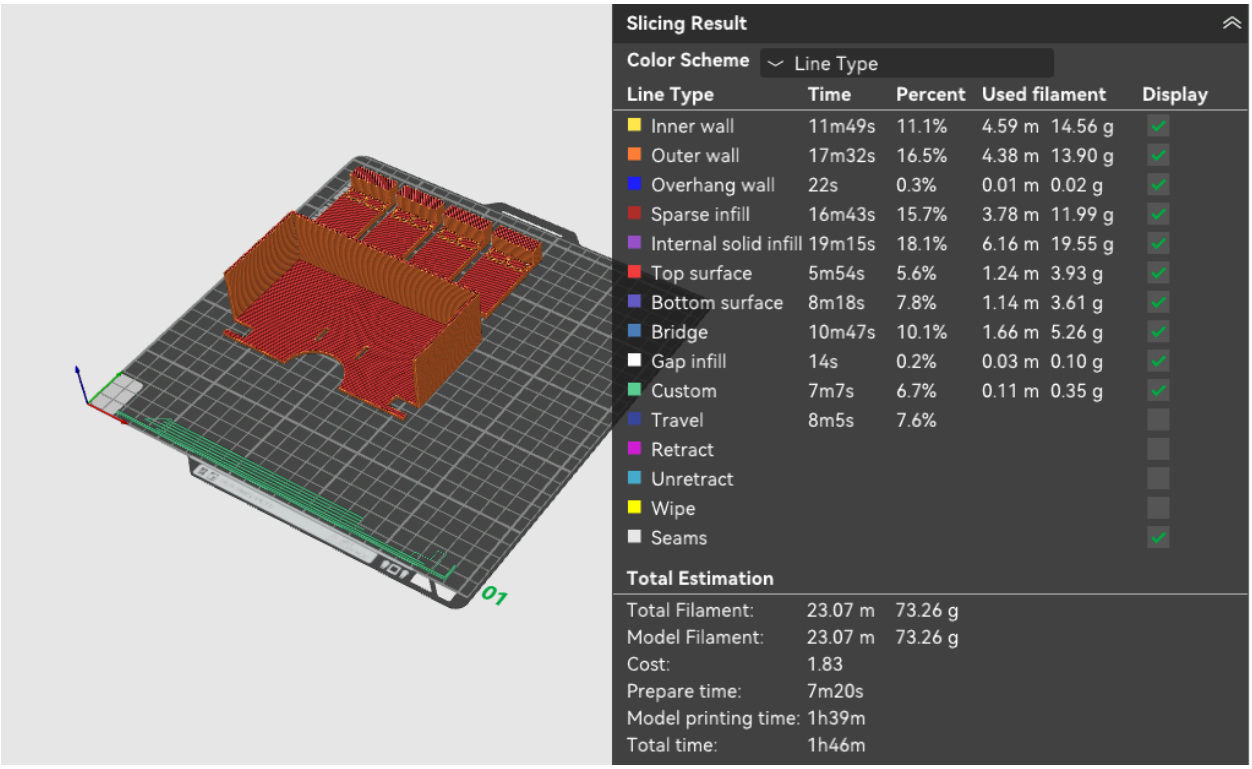
CAD of collection box:



CAD of 1 Hinge:



Volume and Weight Calculations:



Slicing Result

Color Scheme: Line Type

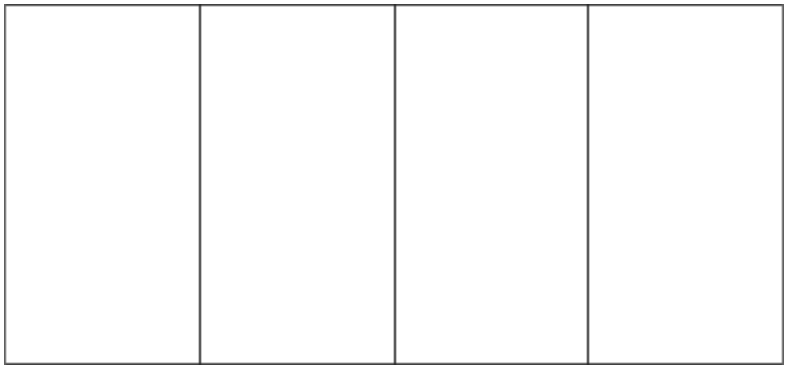
Line Type	Time	Percent	Used filament	Display
Inner wall	11m49s	11.1%	4.59 m 14.56 g	✓
Outer wall	17m32s	16.5%	4.38 m 13.90 g	✓
Overhang wall	22s	0.3%	0.01 m 0.02 g	✓
Sparse infill	16m43s	15.7%	3.78 m 11.99 g	✓
Internal solid infill	19m15s	18.1%	6.16 m 19.55 g	✓
Top surface	5m54s	5.6%	1.24 m 3.93 g	✓
Bottom surface	8m18s	7.8%	1.14 m 3.61 g	✓
Bridge	10m47s	10.1%	1.66 m 5.26 g	✓
Gap infill	14s	0.2%	0.03 m 0.10 g	✓
Custom	7m7s	6.7%	0.11 m 0.35 g	✓
Travel	8m5s	7.6%		
Retract				
Unretract				
Wipe				
Seams				✓

Total Estimation

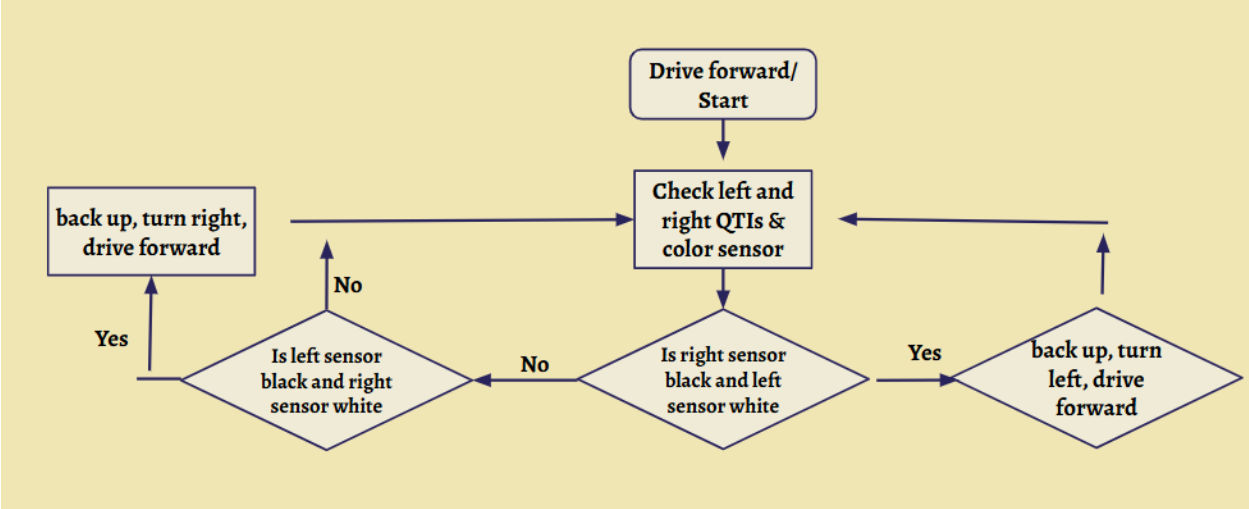
Total Filament:	23.07 m	73.26 g
Model Filament:	23.07 m	73.26 g
Cost:	1.83	
Prepare time:	7m20s	
Model printing time:	1h39m	
Total time:	1h46m	

Total 3D printing price: 73.26*0.4+1 = **$30.30**

Laser Cut DXF:



Appendix D



Appendix E

finalcode.ino

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdlib.h> // Required for the abs() function

// Navigation States
#define SEEK_DIVIDER 0
#define TURN_180     1
#define RETURN_HOME  2
#define FINISHED     3

// Function Prototypes - MUST match the names at the bottom exactly
void initColor();
int getColor();
uint16_t readQTI();
void drive_forward();
void drive_backward();
void pivot_left();
void stop();

volatile uint16_t timerValue = 0;
uint8_t confirmationCount = 0;
int going = 0;

// Color Sensor Interrupt on Port D, Pin 4 (PD4)
ISR(PCINT2_vect) {
    if (PIND & 0b00010000) {
        TCNT1 = 0; // Rising edge: Reset timer
    } else {
        timerValue = TCNT1; // Falling edge: Capture period
    }
}

```

```
int main(void) {
    init(); // Required for Serial and delay() to function
    Serial.begin(9600); // Use serial port
    initColor(); // Setup function for color sensor

    // Setup motor pins, make pins 8, 9, 10, 11 (PB0-PB3) all outputs
    DDRB |= 0b00001111;

    uint8_t state = SEEK_DIVIDER;

    // Averaging initial reading to establish the "Home" color
    long sum = 0;
    int numSamples = 10;

    for (int i = 0; i < numSamples; i++) {
        sum += getColor(); // Add together 10 color readings of home color
        delay(20); // Small delay between samples for independence
    }

    int startColor = sum / numSamples; // Find average -> Home color


```

```
    while (state != FINISHED) {
        // Read right and left QTI readings and color reading
        uint16_t qtiR = readRightQTI();
        uint16_t qtiL = readLeftQTI();
        int currentColor = getColor();

        // --- STATE 0: FIND THE COLOR TRANSITION ---
        if (state == SEEK_DIVIDER) {

            if ((qtiR > 1200) && (qtiL < 1200)) { // RIGHT QTI Hits Black Side Border
                drive_backward();
                delay(200);
                pivot_left();
                Serial.println("Pivot left"); // This was from debugging
                delay(200);
            }
            else if ((qtiR < 1200) && (qtiL > 1200)) { // LEFT QTI Hits Black Side Border
                drive_backward();
                delay(200);
                pivot_right();
                Serial.println("Pivot right"); // This was from debugging
                delay(200);
            }
            else{ // If neither QTI is on a border, just drive forward
                drive_forward();
            }

        }

    }

    // Final infinite loop to ensure it stays stopped
    while(1) {
        stop();
    }
}
```

```
// --- Sensor Logic Functions ---

void initColor() {
    DDRD &= 0b11101111;   // Set Pin 4 (PD4) as Input
    PCICR |= 0b00000100;  // Enable Pin Change Interrupts for Port D
    PCMSK2 |= 0b00010000; // Enable specifically for Pin 4
    TCCR1A = 0;           // Timer1: Normal Mode
    TCCR1B = 0b00000001;  // Timer1: No Prescaler (16MHz)
    sei();                // Global Interrupt Enable
}

int getColor() {
    // Return period in microseconds (ticks * 0.0625 * 2)
    return (int)(timerValue * 0.125);
}

uint16_t readRightQTI() {
    uint16_t count = 0;
    DDRB |= 0b00010000;  // Pin 12 to Output
    PORTB |= 0b00010000; // Drive HIGH
    delay(1);            // Charge capacitor

    DDRB &= 0b11101111;  // Pin 12 to Input
    PORTB &= 0b11101111; // Pull-up OFF

    // Time the discharge until Pin 12 goes LOW
    while ((PINB & 0b00010000) && (count < 5000)) {
        count++;
    }
    return count;
}

uint16_t readLeftQTI() {
    uint16_t count = 0;
    DDRB |= 0b00100000;  // Pin 13 to Output
    PORTB |= 0b00100000; // Drive HIGH
    delay(1);            // Charge capacitor

    DDRB &= 0b11011111;  // Pin 12 to Input
    PORTB &= 0b11011111; // Pull-up OFF

    // Time the discharge until Pin 12 goes LOW
    while ((PINB & 0b00100000) && (count < 5000)) {
        count++;
    }
    return count;
}
```

```
void drive_forward() {
    PORTB &= 0b11110000; // Clear motor pins
    PORTB |= 0b00000101; // Pins 8 & 10 HIGH
}

void drive_backward() {
    PORTB &= 0b11110000;
    PORTB |= 0b00001010; // Pins 9 & 11 HIGH
}

void pivot_left(){
    PORTB &= 0b11110000;
    delay(10);
    PORTB |= 0b00000110; // Left Back and Right Forward
}

void pivot_right(){
    PORTB &= 0b11110000;
    PORTB |= 0b00001001; // Right Back and Left Forward
    delay(100);
}

void stop() {
    PORTB &= 0b11110000; // All motor pins LOW
}

```